

Alberta Buck – Identity Requirements and Cryptographic Receipts

Perry Kundert

2026-03-14

Every operation in the Alberta Buck ecosystem has identity requirements. Some operations require fully public identity (oracle claims, governance votes). Others require only that the participant is a verified identity, without disclosing who they are (BUCK transfers). DeFi integrations (Uniswap pools, lending protocols) introduce contract addresses as counterparties, which also need identities associated with their deployers.

This document tabulates the identity requirements for every operation, then walks through the cryptography of a DeFi swap transaction as a concrete example.

Contents

1	Identity with Anonymity	2
1.1	Revealing Identity	2
1.2	Accounts Belong to Someone	3
1.3	Counterparties Exchange Identities	3
2	Identity Visibility Modes	3
3	Operations Table	3
3.1	Core BUCK (ERC-20) Operations	3
3.2	BuckCredit (ERC-721) Operations	4
3.3	BuckKController (PID Stabilization)	4
3.4	Oracle Operations	4
3.5	DeFi Integration (Uniswap, Lending, etc.)	4
3.6	Identity Management	5
4	Contract Identity: How DeFi Pools Become BUCK Counterparties	5
5	The Receipt Problem: What Can Be Public, What Cannot	6
5.1	The Fundamental Challenge: Proving Re-Encryption Correctness	6
5.2	Homomorphic Equality Check Without Decryption	8
5.3	The <code>approve</code> Function as Identity Handshake	9
5.4	Receipt Construction: What the Contract Emits	10
5.5	The Complete Flow	11

5.6	Summary: What Each Layer Sees	11
6	Cryptographic Receipts: 3-Party ECDH	12
6.1	The Three Keys	12
6.2	Encryption Scheme	12
6.3	Who Can Decrypt	13
7	Play-by-Play: BUCK/USDT Swap on Uniswap	14
7.1	Prerequisites (One-Time Setup)	14
7.2	The Swap Transaction	14
8	Edge Cases and Design Considerations	18
8.1	Flash Loans and Atomic Transactions	18
8.2	Multi-Hop Swaps	18
8.3	Contract-to-Contract Transfers	18
8.4	Wrapping and Bridging	19
8.5	Gas Costs	19
8.6	Privacy of Pool Deployer Identity	19
9	Summary: Identity Requirement by Risk Level	19

1 Identity with Anonymity

The expectation to be able to transact in complete freedom (no one else can prevent it) and anonymity (no one else can identify the parties) is paramount. This is consistent with the expectation that one is innocent until proven guilty. Asking permission from anyone before we can buy, sell or trade is unacceptable to any free people. Anything less turns a free citizen into a serf. If the government can lock your account and freeze you on a park bench at will, how free are you?

However, the expectation that I can trust that my counterparty's identity is *at least* as reliable as the group(s) it is part of is also critical. Allowing arbitrary hostile groups to ravage a community under the cover of darkness has been used throughout history to subjugate free people. If the government can issue identity documents to whomever they wish, and you are forced to trust them, how free are you?

The Alberta Buck enforces these principles.

1.1 Revealing Identity

If I am suspected of a crime, an Alberta court can compel me to decrypt transactions they suspect are related to that crime, and reveal the identity of counterparties I've transacted with. I may resist doing so, and face the consequences: the organization that the Alberta government tasks with overseeing the authenticity of my Alberta identity cannot simply authorized decryption of my transaction counterparties! They can vouch for my (decrypted) identity, just as I can vouch for my daughter's.

Only the participants in a transaction can authorize the decryption of the transaction receipt containing the identity of counterparties and details of the transaction. Thus, the receipt must be encrypted with threshold encryption, such that any *one* of N parties can use their private key to obtain the key to decrypt it ¹

1.2 Accounts Belong to Someone

I may be a citizen of Alberta and have a minor child. My child's identity is vouched by me, and I am vouched by Alberta.

1.3 Counterparties Exchange Identities

Each transaction

2 Identity Visibility Modes

Mode	What on-chain reveals	What off-chain reveals
Anonymous-valid	Address is verified (bit) isVerified(addr) == true	Encrypted receipt: identities of both parties, decryptable only by participants + optional regulatory escrow key
Public	Address is verified + public identity claim on ONCHAINID	Full name, jurisdiction, credentials, track record – world-readable
Contract	Contract address is verified; linked to deployer's identity	Deployer's identity is the contract's identity; receipts encrypted to deployer
Permissionless	No identity check	N/A

3 Operations Table

3.1 Core BUCK (ERC-20) Operations

Operation	Caller identity	Counterparty identity	Receipt	Notes
mint	Anonymous-valid KYC (topic 1)	N/A (self-mint)	Event log	Caller must hold BuckCredits; identity verified but not disclosed on-chain
burn	Anonymous-valid KYC (topic 1)	N/A (self-burn)	Event log	Reduces outstanding balance
transfer	Anonymous-valid KYC (topic 1)	Anonymous-valid KYC (topic 1)	Encrypted 3-party	Both parties verified; receipt encrypted to sender + receiver + escrow
transferFrom	Anonymous-valid KYC (topic 1)	Anonymous-valid KYC (topic 1)	Encrypted 3-party	Spender verified; owner and recipient identities in encrypted receipt
approve	Anonymous-valid KYC (topic 1)	Anonymous-valid KYC (topic 1)	Event log	Approving a spender for transferFrom

¹For Ethereum transactions, N is always 2, but Alberta Buck supports other systems like Holochain so N may be arbitrarily large.

3.2 BuckCredit (ERC-721) Operations

Operation	Caller identity	Counterparty identity	Receipt	Notes
createCredit	Public KYC + insurer (42)	Anonymous-valid KYC (topic 1)	Encrypted to parties	Insurer MUST be publicly identified (their reputation backs the credit). Client may be anonymous-valid.
activate	Anonymous-valid KYC (topic 1)	N/A (self)	Event log	Credit owner activates portfolio.
updateCredit	Public KYC + insurer (42)	N/A	Event log	Insurer updates params; multisig (reappraisal is a public commitment).
transfer (NFT)	Anonymous-valid KYC (topic 1)	Anonymous-valid KYC (topic 1)	Encrypted 3-party	Transferring insurance NFT. Recipient inherits the credit.

3.3 BuckKController (PID Stabilization)

Operation	Caller identity	Receipt	Notes
compute	Permissionless	Event log	Anyone can trigger PID update; minter pays gas. No identity needed.
setGains	Governance	Event log	Governance multisig or vote result. Governance members publicly identified.
addBasketComponent	Governance	Event log	Adding a new commodity to the basket.
setDT	Governance	Event log	Changing the PID update interval.

3.4 Oracle Operations

Operation	Caller identity	Receipt	Notes
Submit claim	Public KYC + reporter (43)	Public log	"I, Perry Kundert, claim Gold was \$2905 at block 19400000." Public identity is the entire point: reputation accrues.
Commit value (L1)	Public KYC + reporter (43)	Event log	Authorized reporters sign the commitment. Signatures are publicly verifiable.
Challenge/dispute	Anonymous-valid KYC (topic 1)	Evidence on-chain	Challenger need not be public; the evidence speaks for itself.
Governance vote (oracle selection)	Anonymous-valid KYC (topic 1)	Vote record	Stake-weighted vote on oracle selection. Vote weight from Buck holdings is public; voter identity may be private.

3.5 DeFi Integration (Uniswap, Lending, etc.)

Operation	Caller identity	Contract identity	Receipt	Notes
Create pool (BUCK/USDT)	Public KYC (topic 1)	Contract registered to deployer's identity	Event log	Pool deployer is publicly Contract address linked to ONCHAINID in IdentityRegistry
Swap BUCK -> USDT	Anonymous-valid KYC (topic 1)	Contract (deployer)	Encrypted 3-party	User sends BUCK to pool Receipt: user + deployer
Swap USDT -> BUCK	Anonymous-valid KYC (topic 1)	Contract (deployer)	Encrypted 3-party	Pool sends BUCK to user
Add liquidity	Anonymous-valid KYC (topic 1)	Contract (deployer)	Encrypted 3-party	User transfers BUCK to pool
Remove liquidity	Anonymous-valid KYC (topic 1)	Contract (deployer)	Encrypted 3-party	Pool transfers BUCK to user

3.6 Identity Management

Operation	Caller identity	Issuer identity	Receipt	Notes
KYC ceremony	Self (no claims yet)	Trusted issuer (publicly identified)	Off-chain encrypted	Participant submits docs to issuer issuer verifies, signs claim.
Register in registry	Registry agent	N/A	Event log	Links wallet address to ONCHAINID
Add public identity claim	Self KYC (topic 1)	Trusted issuer (publicly identified)	On-chain claim	Opts in to public identification for oracle/governance participation.
Revoke claim	Issuer or self	N/A	Event log	Removes a claim (e.g., expired license)
N/M recovery	N-of-M parties (all identified)	N/A	On-chain + off-chain	Transfers identity to new address N of M designated parties agree

4 Contract Identity: How DeFi Pools Become BUCK Counterparties

A Uniswap pool contract is just an address. For BUCK transfers to/from that address to pass `isVerified`, the contract must have an identity in the IdentityRegistry. But a contract can't do KYC. The solution:

The pool deployer's identity becomes the contract's identity.

When someone creates a BUCK/USDT Uniswap pool:

1. Deployer (a verified BUCK participant) calls the Uniswap factory to create the pool.
The factory deploys a new contract at a deterministic address.
2. Deployer (or a registry agent) calls: `identityRegistry.registerIdentity(poolAddress, deployer.onchainId, country)`
This links the pool contract's address to the deployer's ONCHAINID. The pool inherits the deployer's identity claims.
3. Now `isVerified(poolAddress)` returns true. BUCK transfers to/from the pool pass the identity check.

4. Encrypted receipts for pool transactions list the deployer’s identity as the counter-party, because the deployer is the legal operator of the pool.

This means: if you deploy a BUCK DeFi pool, you are the identified legal operator. Your identity is on the line for every transaction through that pool. If a court subpoenas you, you can be compelled to decrypt the receipts (because your key is one of the encryption parties).

5 The Receipt Problem: What Can Be Public, What Cannot

Events are public. Every `emit` on Ethereum is readable by every node. The previous design emitted `ONCHAINID` addresses and ECDH public keys in receipt events – this would create a permanent, public, linkable identity graph of every BUCK transfer. That defeats the entire privacy model.

The core constraints:

1. **The EVM is transparent:** every contract execution, every storage slot, every emitted event is publicly reconstructible. The BUCK contract cannot hold decrypted identity data, even transiently.
2. **The contract is trusted:** the BUCK contract enforces `isVerified` and can verify proofs. What it cannot do is learn secrets.
3. **The identity must be re-encrypted:** when Alice wants to share her identity with Bob (for a receipt), she must produce a ciphertext that Bob can decrypt but nobody else can – and the BUCK contract must be confident this ciphertext contains Alice’s **real** identity, not garbage.
4. **DeFi timing:** in a `transferFrom` call, the router cannot pass receipt data. Identity exchange must happen earlier (at `approve` time).

5.1 The Fundamental Challenge: Proving Re-Encryption Correctness

Alice’s identity `I` is stored encrypted on Holochain, committed on-chain as `C = hash(I, salt)` (a Pedersen commitment or similar). When she wants to share her identity with Bob for a transfer receipt, she must:

1. Decrypt `I` (she has her private key)
2. Re-encrypt `I` under Bob’s public key: `E_bob = Encrypt(I, ECDH(k_alice, K_bob))`
3. Prove to the BUCK contract that `E_bob` encrypts the **same** `I` that `C` commits to – **without revealing `I` to the contract**

This is a **zero-knowledge proof of correct re-encryption**.

5.1.1 What the ZK Proof States

Public inputs (visible to contract and everyone):

```
C          -- Alice's identity commitment (on-chain, from ONCHAINID)
E_bob      -- Alice's identity re-encrypted for Bob
K_bob      -- Bob's public key (from his ONCHAINID or from approve)
```

Private inputs (known only to Alice):

```
I          -- Alice's plaintext identity
salt       -- commitment randomness
k_alice    -- Alice's private key
```

Proof statement:

```
C == Commit(I, salt)
AND E_bob == Encrypt(I, ECDH(k_alice, K_bob))
```

The contract verifies the ZK proof (200K-300K gas for Groth16 SNARK verification). If valid, the contract knows $\sim$ E_bob is a correct re-encryption of the committed identity. The contract never learns I.

5.1.2 Proxy Re-Encryption (PRE): An Alternative

NuCypher's Umbral scheme² provides threshold proxy re-encryption where a proxy transforms a ciphertext from Alice's key to Bob's key without decrypting. Umbral includes non-interactive zero-knowledge (NIZK) proofs of re-encryption correctness.

In the Alberta Buck context:

1. Alice's identity is encrypted under her key: $E_{\text{alice}} = \text{Encrypt}(I, K_{\text{alice}})$
2. Alice computes a re-encryption key: $\text{rk}_{\{A \rightarrow B\}} = \text{ReKeyGen}(k_{\text{alice}}, K_{\text{bob}})$
3. The Holochain hApp (acting as proxy) applies:
 $E_{\text{bob}} = \text{ReEncrypt}(\text{rk}_{\{A \rightarrow B\}}, E_{\text{alice}})$
4. The hApp produces a NIZK proof that re-encryption was correct
5. Alice submits (E_{bob} , proof) to the BUCK contract
6. Contract verifies the proof against E_{alice} (committed on-chain)

The proxy (Holochain hApp) never sees I. It transforms one ciphertext into another without decrypting. The NIZK proof lets the contract verify correctness.

5.1.3 Why Holochain for Proof Generation

ZK proof generation is computationally expensive (seconds to minutes for complex circuits). Holochain provides:

²Umbral: A Threshold Proxy Re-Encryption Scheme. NuCypher. Reference implementation: <https://github.com/nucypher/pyUmbral> Whitepaper: <https://raw.githubusercontent.com/nucypher/umbral-doc/master/umbral-doc.pdf>

- **Trusted code:** the hApp DNA is identified by hash; anyone can verify the exact code that generated the proof. Unlike a random off-chain service, the hApp code is deterministic and auditable.
- **No gas cost:** proof generation runs on the participant's Holochain node, not on Ethereum. The only on-chain cost is proof verification.
- **Agent-centric storage:** Alice's identity data lives on her own source chain, encrypted. The hApp decrypts locally, re-encrypts for Bob, generates the proof – all within Alice's node. Nothing leaves her node except `E_bob` and the proof.

However, Holochain is NOT sufficient alone for trust. Alice runs her own node and could theoretically run modified hApp code. The ZK proof is the trust anchor: it is mathematically verifiable on-chain regardless of what software generated it. Holochain provides the execution environment; the ZK proof provides the guarantee.

5.2 Homomorphic Equality Check Without Decryption

Perry's question: "Is there a homomorphic encryption process by which we can compare an encrypted value as equal, but the party checking for equality cannot decrypt the data?"

Yes. Several approaches:

5.2.1 Pedersen Commitments

$\text{Commit}(v, r) = v \cdot G + r \cdot H$ where G, H are generator points. The commitment hides v (information-theoretically) but binds to it (computationally, under discrete log assumption).

To prove two commitments $C_1 = \text{Commit}(v, r_1)$ and $C_2 = \text{Commit}(v, r_2)$ contain the same value:

Proof: reveal $\text{delta}_r = r_1 - r_2$

Verifier checks: $C_1 - C_2 == \text{delta}_r * H$

The verifier learns nothing about v , only that both commitments bind to the same value. This is efficient (~ 1 EC point addition).

5.2.2 Pedersen + ZK Range Proof (Bulletproofs)

If the identity is a structured value (not just a scalar), Bulletproofs³ can prove properties of committed values without revealing them: range proofs, equality proofs, membership proofs. Verification is $O(\log n)$ and practical on-chain.

³Bulletproofs: Short Proofs for Confidential Transactions and More. Bunz, Bootle, Boneh, Poelstra, Wuille, Maxwell. Overview: <https://tlu.tarilabs.com/cryptography/the-bulletproof-protocols> Applied to Pedersen commitments: <https://www.nccgroup.com/research/on-the-use-of-pedersen-commitments-for-confidential-payments/>

5.2.3 Application to Re-Encryption Verification

The BUCK contract stores Alice's identity commitment: `C_alice = Commit(I, r_alice)`

Alice provides a re-encrypted blob `E_bob` and a ZK proof that the plaintext of `E_bob` is the same `I` committed in `C_alice`.

The contract verifies the proof. It never learns `I`. It learns only that `E_bob` contains the correct identity, re-encrypted for Bob.

5.3 The approve Function as Identity Handshake

The ERC-20 `approve` function is the natural point for identity exchange. Alice is directly calling the BUCK contract – she can provide the re-encrypted identity blob and the ZK proof:

```
function approve(
    address spender,
    uint256 amount,
    bytes    calldata reEncryptedIdentity,    // E_bob: her identity for spender
    bytes    calldata reEncryptionProof      // ZK proof of correct re-encryption
) public returns (bool) {
    _approve(msg.sender, spender, amount);

    // Verify the re-encryption proof against the caller's on-chain commitment
    bytes32 identityCommitment = identityRegistry.identityCommitment(msg.sender);
    require(
        _verifyReEncryption(identityCommitment, reEncryptedIdentity, reEncryptionProof),
        "Invalid re-encryption proof"
    );

    // Store the verified re-encrypted identity for this pair
    _receiptFragments[msg.sender][spender] = keccak256(reEncryptedIdentity);

    // Emit ONLY the re-encrypted blob (opaque to everyone except spender)
    emit IdentityExchange(msg.sender, spender, reEncryptedIdentity);
    return true;
}
```

What the public sees in the `IdentityExchange` event:

- Alice's address approved a spender
- An opaque blob (`reEncryptedIdentity`) that only the spender can decrypt
- **No** ONCHAINID address, **no** public key, **no** linkable identifier

What the contract verified:

- The blob contains Alice’s real identity (ZK proof passed)
- The identity matches her on-chain commitment

What the spender (Bob / pool operator) can do:

- Decrypt the blob using `ECDH(k_bob, K_alice)` to learn Alice’s identity
- Store it for receipt construction

5.3.1 Receiving Transfers: `approve(sender, 0)` Handshake

For an account that wants to **receive** from a specific counterparty:

```
// Alice wants to receive BUCK from Bob (Alice is private-identity):
buck.approve(bob, 0, reEncryptedIdentityForBob, proof)
```

Amount = 0 means no spending authorization. The side-effect: Alice’s re-encrypted identity is now stored and verified for Bob. When Bob later calls `transfer(alice, amount)`, the BUCK contract knows Alice’s identity fragment exists and is verified.

5.4 Receipt Construction: What the Contract Emits

With the corrected design, the BUCK contract emits a **minimal** event that reveals only pseudonymous addresses (which the ERC-20 Transfer event already reveals) plus a reference to the pre-stored identity fragments:

```
event BuckTransferReceipt(
    address indexed from,
    address indexed to,
    uint256 amount,
    bytes32 fromIdentityHash,    // hash of from’s re-encrypted identity
    bytes32 toIdentityHash      // hash of to’s re-encrypted identity
);
```

The `fromIdentityHash` and `toIdentityHash` are hashes of the opaque re-encrypted blobs stored during `approve`. They add **no linkability** beyond what the Transfer event already provides (the addresses themselves are the same).

The actual re-encrypted identity blobs are retrievable:

- From the `IdentityExchange` events (emitted during `approve`)
- From Holochain (stored on the participants’ source chains)

Only the counterparty (who has the private key to decrypt the blob) can reconstruct the full receipt with real identities.

5.5 The Complete Flow

```
function transfer(address to, uint256 amount) public override returns (bool) {
    require(identityRegistry.isVerified(msg.sender), "Sender not verified");
    require(identityRegistry.isVerified(to), "Receiver not verified");
    require(compliance.canTransfer(msg.sender, to, amount), "Compliance");

    _transfer(msg.sender, to, amount);
    compliance.transferred(msg.sender, to, amount);

    // Emit minimal receipt -- no identity data, just hashes of
    // pre-stored re-encrypted blobs
    emit BuckTransferReceipt(
        msg.sender, to, amount,
        _receiptFragments[msg.sender][to],
        _receiptFragments[to][msg.sender]
    );

    return true;
}
```

The `transferFrom` path is identical: the receipt fragment hashes were stored during `approve`, so no additional data passes through the DeFi router.

5.6 Summary: What Each Layer Sees

Layer	What it sees
Ethereum (everyone)	Transfer(from, to, amount) – pseudonymous addresses BuckTransferReceipt – same addresses + opaque hashes IdentityExchange – address + opaque encrypted blob ZK proof verified (pass/fail) – no identity content
BUCK contract	isVerified(from) = true , isVerified(to) = true ZK proof of re-encryption correctness (verified) Hash of re-encrypted identity blob (stored) Never sees plaintext identity
Counterparty (Bob)	Decrypts re-encrypted blob -> learns Alice's identity Constructs full receipt (names, details)
Sender (Alice)	Knows her own identity, knows Bob's (from his approve) Constructs full receipt
Holochain hApp	Generates re-encrypted blob + ZK proof locally Stores encrypted receipts on source chains Code identified by DNA hash (auditable)
Court (with subpoena)	Compels Bob to decrypt his copy of Alice's identity Bob reveals receipt without revealing private key

6 Cryptographic Receipts: 3-Party ECDH

The BUCK contract emits trusted plaintext receipt events (previous section). The wallet layer encrypts these into 3-party ECDH receipts that only the participants can decrypt.

6.1 The Three Keys

For a transfer from Alice to a DeFi pool operated by Bob:

Key	Held by	Purpose
<code>k_s</code> (sender)	Alice	Sender's persistent signing/encryption key
<code>k_e</code> (ephemeral)	Deterministic	Derived from hash of trusted receipt event; reproducible by anyone who knows the event
<code>K_r</code> (receiver)	Bob (deployer)	Pool operator's persistent public key

The ephemeral key `k_e` is derived deterministically from the trusted receipt event (which is on-chain and immutable):

```
// The receipt event fields are all on-chain, all trusted:
receipt_data = abi.encode(
    from, to, amount,
    fromIdentity, toIdentity,
    fromPubKey, toPubKey,
    block.number
)
k_e = keccak256(receipt_data)    // deterministic "private key"
```

6.2 Encryption Scheme

The wallet computes the encryption key via ECDH:

```
// Alice's wallet computes (knows k_s, k_e, K_r):
shared_1 = ECDH(k_s, K_r)        // sender-receiver shared secret
shared_2 = ECDH(k_e, K_r)        // ephemeral-receiver shared secret
encryption_key = keccak256(shared_1 || shared_2)

// Augment the trusted receipt with off-chain identity data:
full_receipt = {
    ...receipt_data,              // from the on-chain event (trusted)
    sender_name: "Alice Johnson", // from ONCHAINID private claim
    receiver_name: "Bob Smith",   // from ONCHAINID claim (via Holochain)
    purpose: "Uniswap V3 BUCK->USDT swap"
}

// Encrypt:
ciphertext = AES-GCM(encryption_key, encode(full_receipt))
```

The encrypted receipt is stored on Holochain (both parties' source chains). A commitment hash can optionally be emitted on-chain:

```
event ReceiptCommitment(
    bytes32 indexed transferHash,    // hash of the BuckTransferReceipt event
    bytes32 receiptHash             // hash of the encrypted receipt
);
```

6.3 Who Can Decrypt

6.3.1 Sender (Alice)

Alice has k_s and can recompute k_e from the on-chain receipt event:

```
k_e = keccak256(receipt_event_data)    // deterministic from on-chain data
shared_1 = ECDH(k_s, K_r)              // she has k_s, K_r from event
shared_2 = ECDH(k_e, K_r)              // she has k_e, K_r from event
encryption_key = keccak256(shared_1 || shared_2)
```

6.3.2 Receiver / Pool Operator (Bob)

Bob has k_r (private key) and reads the sender's public key and the ephemeral public key from the on-chain event:

```
K_e = G * k_e                        // computable from event data
shared_1 = ECDH(k_r, K_s)            // K_s = fromPubKey from event
shared_2 = ECDH(k_r, K_e)            // K_e from event data
encryption_key = keccak256(shared_1 || shared_2)
```

6.3.3 Court-Compelled Decryption (Subpoena of Pool Operator)

A court subpoenas Bob to decrypt a specific transaction receipt:

```
// Court provides: the transaction hash
// Bob looks up: the BuckTransferReceipt event from that transaction
// Bob computes (using his private key k_r):
shared_1 = ECDH(k_r, K_s)            // K_s from event
shared_2 = ECDH(k_r, K_e)            // K_e from event
encryption_key = keccak256(shared_1 || shared_2)
receipt = AES-GCM-decrypt(encryption_key, ciphertext_from_holochain)
```

Bob reveals only the decrypted receipt. He never reveals k_r . He cannot decrypt receipts where he is not a party.

6.3.4 Regulatory Escrow (Optional)

A regulatory authority's public key `K_reg` adds a third shared secret:

```
shared_3 = ECDH(k_e, K_reg)
encryption_key = keccak256(shared_1 || shared_2 || shared_3)
```

The regulator can compute `shared_3` (they have `k_reg`, and `K_e` is deterministic from the on-chain event). But they still need `shared_1` or `shared_2` – requiring cooperation from a transaction participant. No unilateral surveillance.

7 Play-by-Play: BUCK/USDT Swap on Uniswap

Alice holds 1000 BUCK and wants to swap 500 BUCK for USDT. Bob has deployed a BUCK/USDT Uniswap V3 pool.

7.1 Prerequisites (One-Time Setup)

Alice:

1. Completed KYC with trusted issuer (off-chain)
2. Trusted issuer signed claim (topic 1: KYC) on Alice's ONCHAINID
3. Registry agent called `identityRegistry.registerIdentity(`
 `alice_wallet, alice_onchainid, country_code)`
4. `identityRegistry.isVerified(alice_wallet) == true`

Bob (pool deployer):

1. Completed KYC (off-chain)
2. Has KYC claim on his ONCHAINID
3. Is registered in IdentityRegistry
4. Created BUCK/USDT pool via Uniswap factory
 -> pool deployed at address `0xPOOL`
5. Registered pool in IdentityRegistry:
 `identityRegistry.registerIdentity(`
 `0xPOOL, bob_onchainid, country_code)`
6. `identityRegistry.isVerified(0xPOOL) == true`

7.2 The Swap Transaction

7.2.1 Step 1: Alice Approves the Router (Identity Handshake)

Alice's wallet invokes the Holochain `hApp` to prepare the identity exchange, then calls `approve` on the BUCK contract:

```
// 1. Off-chain (Holochain hApp on Alice's node):
//    - Decrypt Alice's identity I from her source chain
//    - Look up the pool operator's (Bob's) public key K_bob
```

```
// - Re-encrypt: E_bob = Encrypt(I, ECDH(k_alice, K_bob))
// - Generate ZK proof: "E_bob encrypts the same I committed on-chain"

// 2. On-chain:
buck.approve(
    uniswapRouter,      // spender (the router, which calls transferFrom)
    500e18,              // amount
    E_bob,              // Alice's identity re-encrypted for Bob
    proof               // ZK proof of correct re-encryption
)
```

The BUCK contract:

1. Sets allowance[alice][uniswapRouter] = 500e18
2. Reads Alice's identity commitment from IdentityRegistry
3. Verifies the ZK proof: E_bob correctly re-encrypts the committed I
4. Stores hash(E_bob) in _receiptFragments[alice][0xPOOL]
5. Emits IdentityExchange(alice, 0xPOOL, E_bob)
 - the event contains ONLY the opaque encrypted blob
 - no ONCHAINID address, no public key, no linkable identifier

This is the critical moment. Alice is directly calling the BUCK contract, so she can provide the re-encrypted identity and ZK proof. By the time `transferFrom` is called later (by the router), the verified identity fragment is already stored.

7.2.2 Step 2: Alice Calls the Router

```
// Alice initiates the swap (standard Uniswap call -- no BUCK-specific data)
uniswapRouter.exactInputSingle(
    tokenIn:  BUCK,
    tokenOut: USDT,
    fee:      3000,          // 0.3% fee tier
    recipient: alice,
    amountIn: 500e18,
    amountOutMinimum: 490e6, // slippage protection
    sqrtPriceLimitX96: 0
)
```

No identity data passes through this call. The router is a standard Uniswap contract that knows nothing about BUCK identity.

7.2.3 Step 3: Router Calls BUCK.transferFrom (Minimal Receipt Emission)

Inside the router, the swap triggers:

```
buck.transferFrom(alice, 0xPOOL, 500e18)
```

The BUCK contract executes (no identity data passes through the router):

```
function transferFrom(address from, address to, uint256 amount) ...
{
    _spendAllowance(from, msg.sender, amount);
    require(identityRegistry.isVerified(from), "Sender not verified");
    require(identityRegistry.isVerified(to), "Receiver not verified");
    require(compliance.canTransfer(from, to, amount), "Compliance");

    _transfer(from, to, amount);
    compliance.transferred(from, to, amount);

    // Emit minimal receipt -- ONLY hashes of pre-stored opaque blobs
    emit BuckTransferReceipt(
        from, to, amount,
        _receiptFragments[from][to], // hash of Alice's E_bob
        _receiptFragments[to][from]  // hash of Bob's E_alice (if exists)
    );

    return true;
}
```

What the public sees: the same from/to/amount as the standard ERC-20 Transfer event, plus two opaque hashes. No ONCHAINID addresses. No public keys. No linkable identity information beyond the pseudonymous Ethereum addresses.

What the contract verified (during approve, earlier): the ZK proof that Alice's re-encrypted identity blob is a correct re-encryption of her committed identity.

7.2.4 Step 4: Wallet Constructs the Full Receipt (Off-Chain)

Alice's wallet (Holochain hApp) observes the `BuckTransferReceipt` event and the `IdentityExchange` events to construct the full receipt:

```
// 1. From on-chain events (trusted, immutable):
transfer_data = {from, to, amount, block.number, tx_hash}

// 2. From IdentityExchange event (emitted during approve):
E_bob = Alice's re-encrypted identity for Bob (opaque blob)

// 3. From Holochain (Alice's source chain):
alice_identity = decrypt(alice_encrypted_identity, k_alice)

// 4. From Bob's IdentityExchange (if Bob also did approve for Alice):
E_alice = Bob's re-encrypted identity for Alice
bob_identity = decrypt(E_alice, k_alice) // Alice can decrypt this
```

```
// 5. Construct full receipt
full_receipt = {
    transfer:      transfer_data,
    sender_name:   alice_identity.name,      // "Alice Johnson"
    receiver_name: bob_identity.name,        // "Bob Smith, DeFi Ops Inc."
    operation:     "Uniswap V3 swap BUCK -> USDT",
    pool:         "0xPOOL",
    timestamp:     block.timestamp
}

// 6. Encrypt the full receipt with 3-party ECDH (see next section)
k_e = keccak256(abi.encode(transfer_data))
shared_1 = ECDH(k_alice, K_bob)
shared_2 = ECDH(k_e, K_bob)
encryption_key = keccak256(shared_1 || shared_2)
ciphertext = AES-GCM(encryption_key, encode(full_receipt))

// 7. Store on Holochain (both parties' source chains)
holochain.store(alice_chain, ciphertext)
holochain.store(bob_chain,  ciphertext)
```

7.2.5 Step 5: Pool Executes the Swap

The Uniswap pool sends USDT to Alice. Since USDT is not an Alberta Buck token, the USDT transfer has no BUCK identity requirements (it follows whatever rules USDT imposes, which is typically none).

7.2.6 Step 6: Completed State

On-chain (public):

- ERC-20 Transfer event: `alice_addr -> 0xPOOL, 500 BUCK`
- `BuckTransferReceipt`: same addresses + two opaque bytes32 hashes
- `IdentityExchange` (from approve): `alice_addr + opaque encrypted blob`
- Swap event: `BUCK/USDT pool, 500 BUCK in, ~495 USDT out`
- ZK proof verification result: pass (no identity content leaked)

Off-chain (Holochain, encrypted):

- Full encrypted receipt on Alice's and Bob's source chains
- Alice's re-encrypted identity blob (decryptable only by Bob)
- Bob's re-encrypted identity blob (decryptable only by Alice)

What anyone can see:

- Two pseudonymous addresses transacted 500 BUCK
- Both addresses pass `isVerified` (they are KYC'd participants)

- Opaque blobs and hashes that reveal nothing without private keys
- A ZK proof was verified (the identity blobs are genuine)

What Alice can see:

- Her own identity (trivially)
- Bob's identity (decrypt his re-encrypted blob with her key)
- Full receipt with names, amounts, purpose

What Bob can see:

- His own identity (trivially)
- Alice's identity (decrypt her re-encrypted blob with his key)
- Full receipt with names, amounts, purpose

What a court can compel (via subpoena of Bob):

- Bob decrypts Alice's blob for the specific transaction
- Reveals Alice's identity for that one receipt
- Bob never reveals his private key

What the public CANNOT see:

- Alice's real name or ONCHAINID
- Bob's real name (as pool operator)
- Any linkable identity across transactions
- The purpose or terms of the transaction

8 Edge Cases and Design Considerations

8.1 Flash Loans and Atomic Transactions

A flash loan borrows and repays in a single transaction. The borrower's address must be verified, and the lending contract must be registered. The receipt covers the full borrow-repay cycle as a single atomic event.

8.2 Multi-Hop Swaps

A swap routed through BUCK -> WETH -> USDT involves two BUCK transfers (BUCK -> pool1, pool1 -> pool2 if intermediate). Each BUCK transfer generates its own encrypted receipt. The router contract must also be registered if it holds BUCK transiently.

8.3 Contract-to-Contract Transfers

When one DeFi contract sends BUCK to another (e.g., a yield aggregator moving BUCK between pools), both contracts must be registered. The receipt identifies both contracts' deployers as the parties. This creates a clear chain of custody: every BUCK movement has an identified legal operator at each end.

8.4 Wrapping and Bridging

If BUCK is wrapped (WBUCK) for use on another chain, the wrapping contract must be registered. The wrap/unwrap generates receipts. On the destination chain, the bridged BUCK may operate under different identity rules (or no identity rules if the destination chain doesn't enforce ERC-3643). This is a known limitation – the Alberta Buck's identity guarantees only hold on chains where the IdentityRegistry is deployed.

8.5 Gas Costs

The corrected design has two gas-consuming operations:

Operation	Cost ($\sim$ gas)	When
<code>approve</code> with ZK proof verify	$\sim$ 200K-300K gas	Once per pair
<code>BuckTransferReceipt</code> event (2 indexed addresses + 3 uint256)	$\sim$ 2,000 gas	Per transfer

The ZK proof verification (`approve`) is expensive but happens once per counterparty pair. Subsequent transfers between the same pair emit only the minimal receipt event (addresses + hashes), which is cheap.

The encrypted receipt itself is stored on Holochain, not on-chain. No encrypted blobs are emitted as Ethereum events.

8.6 Privacy of Pool Deployer Identity

The pool deployer's identity is linked to the pool contract in the IdentityRegistry, but this mapping is on-chain and public. Anyone can look up `identityRegistry.identity(0xPOOL)` and find Bob's ONCHAINID.

If Bob's ONCHAINID has only private claims (KYC but not public identity), then the public knows "0xPOOL is operated by a verified entity" but not WHO that entity is. Bob's identity is revealed only in encrypted receipts.

If Bob has a public identity claim (e.g., "DeFi Ops Inc., Alberta"), then anyone can see who operates the pool. This is likely the expected case for commercial DeFi operators – public identity builds trust and attracts liquidity.

9 Summary: Identity Requirement by Risk Level

Risk level	Operations	Identity mode	Receipt
Highest	BuckCredit creation/update, oracle claims, governance	Public	On-chain event + public record
High	Pool deployment, compliance changes	Public	On-chain event
Medium	BUCK transfer, DeFi swap, add/remove liquidity	Anonymous-valid	Encrypted 3-party ECDH receipt
Low	BUCK burn, credit activation	Anonymous-valid	On-chain event
None	BuckK compute, read oracle, view balances	Permissionless	On-chain event